Week 06

Activity 01

1. Cost of Postage

```
def cost(ounces):
    return 0.05 + (0.10 * ceil(ounces - 1))


def ceil(ounces):
    if int(ounces) != ounces:
        return int(ounces + 1)  # Fixed indentation
    else:
        return ounces  # Fixed indentation


def getInput():
    'Get the weight of postage from user'
    return float(input("Enter number of ounces: "))


def displayOutput(cost):
    print("Cost: ${0:.2f}".format(cost))


def main():
    # Input --> Process --> Output
```

```
    ounces = getInput()
    postageCost = cost(ounces)
    displayOutput(postageCost)

main()
```

2. Credit Card Payment

```
# Define the finance charge rate as a constant
RATE = 0.015  # 1.5% finance charge

def calculate(oldBalance, charges, credits):
    'Calculate new balance and minimum payment'
    # Correct formula: Old balance + charges + finance charge - credits
    # Finance charge = oldBalance * RATE
    newBalance = oldBalance + charges + (oldBalance * RATE) - credits

    # Calculate minimum payment based on new balance
    if newBalance <= 20:
        minPayment = newBalance  # Pay full balance if $20 or less
    else:
        minPayment = 20 + 0.10 * (newBalance - 20)  # $20 + 10% of amount above $20

    return newBalance, minPayment
```

```
def getInput():
    'Get credit card statement information from user'
    oldBalance = float(input("Enter old balance: "))
    charges = float(input("Enter charges: "))
    credits = float(input("Enter credits: "))
    return oldBalance, charges, credits

def displayOutput(newBalance, minPayment):
    'Display calculated statement information'
    print("New Balance: ${0:.2f}\nMinimum Payment: ${1:.2f}".format(newBalance,
minPayment))

def main():
    # Input --> Process --> Output
    oldBalance, charges, credits = getInput()
    newBalance, minPayment = calculate(oldBalance, charges, credits)
    displayOutput(newBalance, minPayment)

# Program execution
main()
```

3. Mortgage Calculation

```
# Mortgage Calculator

def calculate(annualRate, monthlyPayment, beginningBalance):
    # Convert annual rate to monthly decimal rate
    monthlyRate = (annualRate / 12) / 100

    # Calculate interest, principal reduction, and new balance
    interestPaid = beginningBalance * monthlyRate
    reductionPrincipal = monthlyPayment - interestPaid
    endBalance = beginningBalance - reductionPrincipal

    return interestPaid, reductionPrincipal, endBalance

def getInput():
    # Get user input
    annualRate = float(input("Enter annual rate of interest: "))
    monthlyPayment = float(input("Enter monthly payment: "))
    beginningBalance = float(input("Enter beg. of month balance: "))
    return annualRate, monthlyPayment, beginningBalance

def displayOutput(interestPaid, reductionPrincipal, endBalance):
    # Display results with currency formatting
    print("Interest paid for the month: ${0:,.2f}".format(interestPaid))
    print("Reduction of principal: ${0:,.2f}".format(reductionPrincipal))
```

```
    print("End of month balance: ${0:,.2f}".format(endBalance))


def main():
    # Input --> Process --> Output
    annualRate, monthlyPayment, beginningBalance = getInput()
    interestPaid, reductionPrincipal, endBalance = calculate(annualRate, monthlyPayment,
beginningBalance)
    displayOutput(interestPaid, reductionPrincipal, endBalance)


main()
```

4. Wilson’s Theorem

```
# Wilson's Theorem Prime Number Checker
# Determines if a number is prime using Wilson's Theorem:
# n is prime if and only if (n-1)! + 1 is divisible by n


def factorial(n):
    # Calculate factorial of a number
    if n == 0 or n == 1:
        return 1
    else:
        result = 1
        for i in range(2, n + 1):
            result = result * i
```

```
        return result

def isPrime(n):
    # Determine if a number is prime using Wilson's Theorem
    # Wilson's Theorem: n is prime if (n-1)! + 1 is divisible by n
    if n <= 1:
        return False

    # Calculate (n-1)! + 1
    wilsonValue = factorial(n - 1) + 1

    # Check if n divides (n-1)! + 1
    if wilsonValue % n == 0:
        return True
    else:
        return False

def getInput():
    # Get number from user
    number = int(input("Enter a number to check if it's prime: "))
    return number

def displayOutput(number, result):
    # Display whether the number is prime or not
    if result:
        print("{0} is a prime number".format(number))
```

```
    else:
        print("{0} is not a prime number".format(number))

def main():
    # Input --> Process --> Output
    number = getInput()
    result = isPrime(number)
    displayOutput(number, result)

# Program execution
main()
```

Activity 02

1. Semester Grade

```
# Semester Grade Calculator
# Calculates semester grade based on midterm and final exam scores
# Final exam counts twice as much as midterm exam

def ceil(number):
    # Round a number up to the next integer
    if int(number) != number:
        return int(number + 1)
```

```
    else:
        return int(number)

def semesterGrade(midterm, finalExam):
    # Calculate semester grade based on weighted average
    # Final exam counts twice as much as midterm
    weightedAverage = (midterm + (2 * finalExam)) / 3

    # Round up to the nearest whole number
    roundedAverage = ceil(weightedAverage)

    # Assign letter grade based on rounded average
    if roundedAverage >= 90:
        return 'A'
    elif roundedAverage >= 80:
        return 'B'
    elif roundedAverage >= 70:
        return 'C'
    elif roundedAverage >= 60:
        return 'D'
    else:
        return 'F'

def getInput():
    # Get numeric grades from user
    midterm = float(input("Enter midterm grade: "))
```

```
    finalExam = float(input("Enter final exam grade: "))
    return midterm, finalExam

def displayOutput(grade):
    # Display the semester grade
    print("Semester grade: {0}".format(grade))

def main():
    # Input --> Process --> Output
    midterm, finalExam = getInput()
    grade = semesterGrade(midterm, finalExam)
    displayOutput(grade)

# Program execution
main()
```

2. Planets

```
# Planets Program
# Displays planets in descending order by surface area

def sortBySurfaceArea(planets):
    # Sort planets by surface area in descending order
    # Use lambda function to sort by the second element (surface area)
```

```
    # reverse=True sorts in descending order
    sortedPlanets = sorted(planets, key=lambda x: x[1], reverse=True)
    return sortedPlanets

def getPlanetNames(sortedPlanets):
    # Extract just the planet names from the sorted list
    planetNames = []
    for planet in sortedPlanets:
        planetNames.append(planet[0])
    return planetNames

def displayOutput(planetNames):
    # Display planet names in descending order by surface area
    print("Sorted by surface area in descending order:")
    # Join planet names with spaces for clean output
    print(" ".join(planetNames))

def main():
    # Input --> Process --> Output
    Planets = [('Mercury', 75, 1), ('Venus', 460, 2), ('Mars', 140, 4),
               ('Earth', 510, 3), ('Jupiter', 62000, 5), ('Neptune', 7640, 8),
               ('Saturn', 42700, 6), ('Uranus', 8100, 7)]

    # Process: Sort planets by surface area
    sortedPlanets = sortBySurfaceArea(Planets)
```

```
    # Process: Extract planet names
    planetNames = getPlanetNames(sortedPlanets)

    # Output: Display results
    displayOutput(planetNames)

# Program execution
main()
```

tkinter_tutorial

1. Quick Exercises

I. Write fahrenheit_to_celsius(f) that returns °C.

```
# tmpConverter.py
# Temperature Converter

# Write fahrenheit to celsius(f) that returns °C.
def fahrenheit_to_celsius(f):
    'Converts fahrenheit to celsius'
```

```
    return (f - 32) * 5 / 9


def getInput():
    'Get fahrenheit'
    return float(input("Enter temperature in fahrenheit: "))


def displayOutput(f, c):
    # Fixed formatting: use .format() with both values
    print("{0:.2f} Fahrenheit = {1:.2f} Celsius".format(f, c))


def main():
    # Input --> Process --> Output
    f = getInput()
    c = fahrenheit_to_celsius(f)
    displayOutput(f, c)


main()
```

II. Write `is_even(n)` that returns `True` if `n` is even.

```
# Even Number Checker


def is_even(n):
    # Returns True if n is even
```

```
    return n % 2 == 0


def getInput():
    # Get number from user
    number = int(input("Enter a number: "))
    return number


def displayOutput(number, result):
    # Display whether the number is even or odd
    if result:
        print("{0} is even".format(number))
    else:
        print("{0} is odd".format(number))


def main():
    # Input --> Process --> Output
    number = getInput()
    result = is_even(number)
    displayOutput(number, result)


# Program execution
main()
```

III. Write `triangle_area(b, h=1)` with default height 1.

```
# Triangle Area Calculator


def triangle_area(b, h=1):
    # Calculate area of triangle with default height of 1
    return (b * h) / 2


def getInput():
    # Get base from user
    base = float(input("Enter base of triangle: "))


    # Get height from user (optional)
    height = input("Enter height (press Enter for default 1): ")


    if height == "":
        return base, None  # Use default height
    else:
        return base, float(height)


def displayOutput(base, height, area):
    # Display triangle area
    if height is None:
        print("Triangle area (base={0:.2f}, height=1.00): {1:.2f}".format(base, area))
    else:
        print("Triangle area (base={0:.2f}, height={1:.2f}): {2:.2f}".format(base, height, area))
```

```
def main():
    # Input --> Process --> Output
    base, height = getInput()

    if height is None:
        area = triangle_area(base)  # Uses default height of 1
    else:
        area = triangle_area(base, height)

    displayOutput(base, height, area)

# Program execution
main()
```